OAuth Whiteboard — System Architecture & Workflow Flowcharts

Google OAuth 2.0 Integration & On-Demand Search Console Indexing Workflow

---

1. High-Level System Architecture & Component Flow

The application follows a clean 4-tier asynchronous architecture with strict defense-in-depth isolation:

Tier	Component & Responsibility
Client Tier	Browser / API consumer interacting via HTTP REST API and Bearer / Header tokens.
FastAPI Layer	Asynchronous routing, dependency injection, SSRF filtering, and exception envelopes.
Security Layer	Fernet AES-128-CBC + HMAC encryption at rest, MultiFernet key rotation, and HMAC-SHA256 state signing.
Database Layer	SQLAlchemy 2.0 Async (PostgreSQL asyncpg / SQLite aiosqlite fallback) with unique user/provider constraints.
Google APIs	OAuth 2.0 token services, Google Indexing API (urlNotifications:publish), and Search Console APIs.

2. Google OAuth 2.0 Connection & Callback Workflow

• **Step 1: Initiation (GET /oauth/google/connect):** Client calls connect endpoint. Backend generates HMAC-SHA256 signed state encoding user_id, 16-byte nonce, timestamp, and purpose. Issues 302 redirect to Google with access_type=offline and prompt=consent.

• **Step 2: User Consent:** User authenticates on Google's consent screen. Google issues a single-use authorization code and redirects back to the registered callback URI.

• **Step 3: State Validation & Anti-CSRF:** Backend receives code & state. Validates: (1) HMAC signature matches, (2) timestamp <= 600s, (3) state user_id matches requesting user (mitigating mix-up attacks), and (4) nonce has never been used (eliminating replay attacks).

• **Step 4: Server-Side Code Exchange:** Backend sends a POST request with client_id, client_secret, code, and redirect_uri directly to https://oauth2.googleapis.com/token. Plaintext tokens are NEVER exposed to client.

• **Step 5: OIDC Identity Verification:** Backend queries https://openidconnect.googleapis.com/v1/userinfo to verify user sub and email, ensuring unverified claims are never accepted.

• **Step 6: Authenticated Encryption & Upsert:** Tokens are encrypted using Fernet AES-128-CBC + HMAC. If Google omits a refresh token on re-consent, the existing valid refresh token is preserved. Record is stored with status ACTIVE.

3. On-Demand URL Indexing Workflow (Google Indexing API)

• **Step 1: Inbound Request (POST /indexing/request):** Client submits URL and notification type (URL_UPDATED or URL_DELETED).

• **Step 2: Strict SSRF Filtering:** URL scheme must be HTTPS. Rejects localhost, IPv4/IPv6 private subnets (RFC 1918), cloud metadata IPs (169.254.169.254), and performs DNS resolution checks to prevent DNS rebinding.

• **Step 3: Pre-Emptive Expiry Buffer:** TokenService checks if token expires within 300 seconds (5 minutes). If expired or close to expiry, automatic token refresh is triggered before dispatching the API call.

• **Step 4: Async Mutex Concurrency Lock:** An asyncio.Lock per user_id ensures concurrent indexing requests execute only ONE refresh call to Google, preventing race conditions and token invalidation.

• **Step 5: Google Indexing API Dispatch:** Request is sent via HTTPX to https://indexing.googleapis.com/v3/urlNotifications:publish with Bearer decrypted_token.

• **Step 6: Single-Retry on 401 Unauthorized:** If Google returns 401, the service forces token refresh and retries the request exactly once. If it fails again, it aborts cleanly, preventing infinite loops.

• **Step 7: Safe Normalized Response:** Returns safe delivery status without leaking tokens or claiming guaranteed indexing.

4. Disconnect, Revocation & Password Changes

• **Remote Revocation (DELETE /oauth/google/disconnect):** POST to https://oauth2.googleapis.com/revoke with form-encoded token=PLAINTEXT_TOKEN.

• **Network Failure Resilience:** If Google revocation endpoint times out or fails, local disconnection is still guaranteed: credentials in database are overwritten with null, status set to REVOKED, and remote_revoked: false is accurately reported.

• **Google Password Change Behavior:** When a user revokes consent or changes credentials, Google token endpoint returns invalid_grant. TokenService detects this, flags the connection as reauth_required, aborts retry loops, and prompts user to reconnect.